Recitation 5

Topics

- Design interacting classes using
    - Association
    - Composition
- Delegation
- Copy control: destructor and copy constructor.
- Nesting classes
- Output operators

Attachments

- rec05_start.cpp
- output.txt
- Code Reading Question.pdf

Task

In this lab you will be modeling a little part of the School of Engineering CS lab administration.

You are being asked to write software so that the University programmers can model the way your lab instructors keep up with your grades.

We've reduced this problem a great deal so that there is not too much for you to do. The complete set of code would be really long and would require days or weeks of analysis.

Remember that the focus of the lab is to practice encapsulation, data hiding, association and delegation. The first two are basic tenets of OOP. Delegation means that if one object holds a piece of data and you want something done with that data, then you *ask the object to do it for you.*

First checkout

You will be defining four classes. It would be good to check out with a TA once you have

- defined the four classes, i.e. what are their **fields**? If the classes are nested, should they be nested publicly or privately?
- written their **constructors**
- written their **output operators**

Do this *before* you worry about defining any other methods. This way you will be sure that you are on the right track to begin with.

Of course, uncomment the test code for task one and make sure it builds and runs reasonably. [*Always* test your code.]

Each semester a set of lab sections is created and a lab instructor is assigned to each lab.

Lab instructors have names (do you know your lab worker's name?).
To make the problem less complicated, for this problem no lab worker will be assigned to more than one lab.
Also assume that lab workers never make any mistakes when entering grades for students, so your code does not have to check for such a possibility.

It's the worker's job to enter grades for their students each week and to print out a gradesheet showing all the students and their grades for each lab.

This is really only about lab grades, so here's the process you must implement in your classes:
Grades should be kept separately for each week - so each week the lab worker adds the student's score for that week to the current grade record for the section he is teaching.
Each ***student lab record*** consists only of the student's name and their weekly grades. [Assume that no two students in the University have the same name (see, we reduced the problem even more!] The value -1 is used to indicate an absence from a lab - can you write the code so that everyone is absent before the first grade is entered? (this is something that is often done for consistency).

A ***lab section*** will be the "container" for student lab records in this problem.
One lab section object will contain and manage the student lab records for a specific section. So there's a one-to-one connection between a lab worker and a section (because lab workers in this reduced problem work only one lab each). A lab section is kind of like an accounting package or spreadsheet in that the lab worker uses it to keep

his students' grades. The University generates a file for each lab section that has the names of all the students who in that lab. Each lab section is "filled" with students from the corresponding file. There will be one student lab record for each student in the file and all the student lab records for a section will be stored in a lab section object. There's no maximum number of students in a lab section. How should we represent this? Clearly a vector is a good idea. And we *could* (but won't) make it a vector of student records. Why won't we? Well, for this lab, we want you to allocate the student records on the heap. So, what will the vector be holding? *Pointers*!

Each section also has a thing called a ***time slot*** which has the day of the week and the hour the lab section begins. Make this be a class with the day of the week being a string. and the hour an int (or more correctly an unsigned). For simplicity times are provided to the constructor in 24-hour time, i.e. 4pm would be represented as 16. For display, we show the times with AM or PM instead of 24-hour notation since most people find that more readable.

Displaying: the programmers we are working for have requested both the section and the lab instructor can be displayed using output operators (see the sample output below). For consistency, we will also display the Student and Timeslot objects using their own output operators.

I**mplement output operators for all four classes**. If you know how, it would be good to define them outside of the class definitions. But we won't complain if for this lab, you choose to put the definitions for the output operators inside their respective classes. (They are still going to be marked as **friend** either way, right?

The programmers who hired you also want to reuse the Sections and lab workers after they have been created. The programmers (your bosses) want you to provide a **reset** function for the section object that clears out the data for the student records. The time slot remains the same (forever).

Constructors: Should a lab worker be created without specifying a name? Or a section? Should a section be created with a properly initialized time slot? As always, we should only have constructors that set up the fields appropriately.

You are writing code for the programmer who will use your classes to model the labs at Poly.
We will provide you with a testing program that will show you what sorts of things need to be done and how the programmers you are working for want your classes to work: the programmer interface you must provide is shown in the test code.

[In the exams, you may get the same sort of thing: a `main()` function will be given to you and you'll have to write the classes that implement what's done in `main()`. Activities that happen in the "real world" (or some tiny version of it like the lab record keeping) will be modeled by a programmer using your classes.

The classes

To summarize: you need classes to represent

- **Lab worker**. It needs to know what section it is teaching, but the section can exist prior to the lab worker.
- **Section**. Knows the time and student records.
    - The student records are *on the heap*.
    - The Section does *not* know the LabWorker, even though that would seem reasonable. Why not? Because that would introduce additional problems we haven't covered yet.
- **Time slot**. This object is stored directly in the Section, i.e. it is "contained" in the Section.
- **Student record**. Should hold a vector of the 14 grades for the semester. The student records are maintained on the heap.

Note: The last two classes will not be directly accessed by the user's code, so we will **nest them privately**.

Association vs. Composition

Consider the relationships among our classes. A Section instance is *composed* of, among other things, a TimeSlot. The Section cannot exist without the TimeSlot. In fact, they both come into existence together and late cease existing together.

The bond or *association* between a Lab worker and a Section is quite different. They exist completely indepently and the bond can be made or broken at any point.

These two approaches to establishing connections between different kinds of objects are very common. The nature of an association, such as the spousal relationship from lecture, makes pointers an obvious tool for implementation. Composition, on the other hand, is more likely represented, as we have here, by specifying to type have a field of the other type.

The software design literature goes into much more detail about the different relationships that might exist between classes, but this will suffice for our needs.

Second Checkout

We encourage you to get checked out on the work above before implementing Copy Control. And, of course, uncomment the appropriate parts of the test code, then build and run.

Copy Control

What should happen when a Section object "ceases to exist" (i.e. goes out of scope if it is on the stack, or is deleted if it is on the stack)? All of its Student record objects (which all exist on the heap) need to be freed up. Note in our output, you see messages saying that the Section "is being deleted" and then that each of the students in that section is being deleted.

Where are these print statements in the code? They are not in the test code that we gave you. Instead they are in the Section **destructor**. Naturally we wouldn't normally have such print statement in a destructor, but it is nice when we are debugging, or just trying to understand what's going on.

What does the **copy constructor** need to do? It is initializing a new object to look just like the original. That means that when we write it, our code has to copy *all* the fields in the original, including the name and time, not just the student records. Remember, the Section is holding a vector of *pointers to the heap.* You are to do a *deep copy*, so you need to allocate a new Student record for each one that you are copying.

Assignment operator? We are **not** asking you to write an assignment operator, though **it is of course part of copy control** and normally if you are going to need one, you almost certainly need the other two. (They are sometimes referred to as the **Big 3**.)

All test code will be uncommented by now, and you will have verified that all of your output matches ours.

Submit

Remember to try the **Code Reading Question**!

- A single file, **rec05.cpp**